

CodePrep AI

Project Overview — AI-Powered Coding & Placement Preparation Platform

Vision: Build a free, open-source, self-hosted platform that takes students from learning programming and DSA to coding practice, assessments, company preparation, interviews, and placement readiness.

1. Product Scope & Target Users

Students: learn concepts, follow roadmaps, solve coding problems, take assessments, track progress, prepare for companies and interviews, and use AI for guided learning. **Faculty:** create courses, problems, assignments and assessments, manage batches, and analyze performance. **College Admin:** manage departments, faculty, students and placement-readiness reports. **Platform Admin:** manage users, content, companies, assessments, judge infrastructure, certificates and analytics.

2. Core Modules

Module	Key Capabilities
Authentication & RBAC	Registration/login, JWT, refresh tokens, roles: Student, Faculty, College Admin, Content Editor, Super Admin.
Student Dashboard	Progress, streaks, problems solved, assessment scores, recommendations, weak topics and recent activity.
Learning Management	Courses → modules → topics → lessons → examples → practice → assessments; progress tracking.
DSA Roadmaps	Structured paths covering programming basics, arrays, strings, searching, sorting, recursion, linked lists, trees, graphs and DP.
Coding Platform	Problem library, difficulty/topic/company filters, Monaco Editor, starter code, hints, editorials and solution support.
Online Judge	Secure Docker-based execution for Java, Python, C++ and JavaScript with time/memory/output limits.
Assessment Engine	MCQ, multiple-select, coding, debugging, output prediction, SQL, aptitude, logical reasoning and verbal tests.
Placement Preparation	Company-wise preparation, previous questions, coding rounds, technical/HR interview preparation and mock tests.
Faculty & College	Batch management, assignments, scheduled tests, student analytics, department reports and placement-readiness dashboards.
AI Layer	Local AI tutor, hints, code explanation/debugging, question generation, interview simulation and personalized recommendations.

3. Recommended 100% Free/Open-Source Technology Stack

Layer	Technology
Frontend	Next.js + TypeScript + Tailwind CSS + shadcn/ui + Monaco Editor + TanStack Query + Zustand + Zod
Backend	Python + FastAPI + SQLAlchemy + Pydantic + JWT authentication
Data	PostgreSQL + Redis
Async Processing	RabbitMQ
Code Judge	Docker isolated workers; language runtimes for Java/Python/C++/JavaScript
Storage	MinIO (S3-compatible self-hosted object storage)
AI	Ollama + locally hosted open-source coding/reasoning models
Testing	Pytest + Playwright + Testcontainers
DevOps	Git, GitHub, Docker Compose, Nginx
Monitoring	Prometheus + Grafana; optional self-hosted Sentry/PostHog

CodePrep AI — Architecture, Requirements & Delivery Plan

4. High-Level Architecture

Users → Next.js Frontend → FastAPI Backend → PostgreSQL / Redis / RabbitMQ / MinIO

RabbitMQ routes coding submissions to isolated **Judge Workers** → **Docker Sandboxes** → **Test Cases** → **Results**.

The AI layer connects the backend to **Ollama** for local tutoring, code assistance, interview simulation and content generation.

Monitoring is provided through Prometheus/Grafana.

5. Critical Functional Requirements

- **Problem management:** problems must support metadata, topics, companies, difficulty, examples, constraints, hints, editorials, starter code, test cases, time limits, memory limits and language-specific solutions.
- **Judge security:** student code must never execute inside the main API process. Use isolated containers, CPU/memory/time/output/process limits, restricted filesystem access, no network access and non-root execution.
- **Progress tracking:** record views, attempts, runs, accepted submissions, topic progress, course completion, assessment history, streaks and recommendations.
- **Assessments:** configurable sections, duration, marks, negative marking, randomization, attempt limits, automatic evaluation and detailed result analytics.
- **Multi-tenant college model:** Organization → Department → Batch → Faculty/Students, with strict role-based access and organization-level data isolation.
- **AI governance:** AI-generated questions/content enter a review workflow before publishing; AI hints should encourage learning rather than reveal complete solutions by default.
- **Audit & security:** role-based authorization, input validation, rate limiting, audit logs, secure secrets, backups and protected administrative actions.

6. Core Data Model

Identity: users, roles, organizations, departments, batches. **Learning:** courses, modules, topics, lessons, progress. **Coding:** problems, problem_topics, problem_companies, test_cases, solutions, submissions, submission_results. **Assessment:** assessments, questions, attempts, answers. **Placement:** companies, interview_questions, mock_tests. **Engagement:** streaks, badges, bookmarks, notifications. **Credentials:** certificates and verification records.

7. MVP — Build This First

Stage	Deliverables
1. Foundation	Repository, Docker Compose, Next.js, FastAPI, PostgreSQL, migrations, environment configuration.
2. Identity	Registration/login, JWT, roles, protected routes and student profile.
3. Learning	Courses, topics, lessons, DSA roadmap and progress tracking.
4. Coding	Problem library, filters, problem page, Monaco Editor, run/submit workflow.
5. Judge	RabbitMQ, worker service, Docker sandbox, test cases, verdicts and submission history.
6. Assessment	MCQ + coding tests, timer, scoring, results and analytics.

8. Post-MVP Roadmap

Phase 2: company-wise preparation, aptitude, SQL, interview preparation, leaderboards, badges and certificates. **Phase 3:** faculty portal, college organizations, batches, assignments, scheduled assessments and placement analytics. **Phase 4:** Ollama-powered AI tutor, debugger, interviewer, question generator and recommendation engine. **Phase 5:** load testing, judge scaling, observability, backups, security hardening and production deployment.

9. Autonomous Development Strategy

Recommended coding agent: OpenHands with Ollama and a locally hosted coding model. Use an incremental workflow rather than asking the agent to build the entire product in one instruction: **PRD** → **Architecture** → **Database** → **Foundation** → **Auth** → **Learning** → **Coding** → **Judge** → **Assessments** → **Placement** → **College** → **AI** → **Testing** → **Security** → **Deployment**. Every stage should include implementation, unit/integration/UI tests, security checks and a clean Git commit before the next stage.

10. Product Differentiator

Core journey: Learn → Practice → Assess → Identify Weakness → Personalized Practice → Company Preparation → Interview → Placement Readiness. The product should not be positioned as only another coding-problem clone. Its strongest opportunity is combining coding practice, structured DSA learning, placement preparation, college training and local AI assistance in one self-hosted ecosystem.

Final stack: Next.js + FastAPI + PostgreSQL + Redis + RabbitMQ + Docker + MinIO + Ollama | **Development model:** modular monolith with isolated judge workers.